This technical report provides the detailed pseudocode of the proposed greedy algorithms and the proofs of the theoretical results. Algorithm 1 (Followers) describes the procedure for computing the follower set induced by an anchored edge. Algorithm 2 (GIA) presents the greedy incremental anchoring strategy, which iteratively selects the edge–weight increment pair with the largest follower gain per unit budget. Algorithm 3 (GPA) accelerates GIA by using candidate reduction, upper-bound pruning, and intermediate-result reuse, while preserving the same greedy selection criterion.

A Pseudo code of Followers

Algorithm 1: Followers(G, e, δ_e, s, C_s)

```
PQ ← queue(); // sort by coreness-layer ascending order
F ← ∅;
w(e) ← w(e) + δ_e;
for each x ∈ {u, v} if x ∉ C_s do
    PQ.add(x);
while PQ ≠ ∅ do
    x ← PQ.pop();
    w⁺(x) ← SupportWeightedDegree(G, x);
    if w⁺(x) ≥ s then
        F ← F ∪ {x};
        for each y ∈ N(x, G) and x ≺ y do
            if y ∉ C_s and y ∉ F then
                PQ.add(y);
    else
        Q ← queue();
        Q.push(x);
        while Q ≠ ∅ do
            y ← Q.pop();
            if y ∈ F then
                F ← F \ {y};
            for each z ∈ N(y, G) ∩ F do
                w⁺(z) ← w⁺(z) − w(y, z);
                if w⁺(z) < s then
                    Q.push(z);
w(e) ← w(e) − δ_e;
return F
```

Algorithm description. Algorithm 1 computes the follower set induced by an anchored edge via a localised simulation of the s-core peeling process. Starting from the anchored endpoints, the algorithm explores only nodes that are reachable through neighbours with higher coreness–layer pairs, following the upstair paths determined by the onion-layer structure. A node is retained as a follower if its support weighted degree meets the s-core threshold; otherwise, it is discarded. Discarding a node may reduce the support of neighbouring retained followers, potentially triggering further removals. This cascading update exactly mirrors the behaviour of the s-core decomposition, but is restricted to the structurally affected region. The process terminates when no further admissible nodes remain, and the retained follower nodes constitute the final follower set.

B Pseudo code of GIA

Algorithm description. Algorithm 2 presents the baseline greedy incremental anchoring procedure of GIA. Starting from the initial s-core, the algorithm iteratively evaluates candidate edges whose endpoints are not both contained in the current s-core. For each candidate edge, GIA first checks budget feasibility using the lower-bound increment $\tilde{\delta}_e$ and performs a single follower computation with $\delta_{\text{test}} = \min\{\overline{\delta}_e, b - sum\}$. If followers are found, it computes the MEI δ_e from the resulting follower set and evaluates the corresponding follower ratio. At each iteration, the edge achieving the largest follower ratio is selected, its weight is increased by δ_e, and the s-core is updated. This process repeats until the budget is exhausted or no feasible positive-gain candidate remains, yielding a greedy baseline for comparison.

Algorithm 2: GIA(G, b, s)

```
G' ← G, A ← ∅, sum ← 0;
C_s ← s-core(G');
while sum < b do
    E* ← C(V, 2) \ C(C_s, 2);
    e* ← none, δ* ← 0, max_FR ← 0;
    for each e = (u, v) ∈ E* do
        δ̃_e ← s − min{c(u), c(v)};
        if sum + δ̃_e ≤ b then
            δ̄_e ← s − w_G'(e);
            δ_test ← min{δ̄_e, b − sum};
            F(e, δ_test) ← Followers(G', e, δ_test, s, C_s);
            if F(e, δ_test) ≠ ∅ then
                H ← C_s ∪ F(e, δ_test);
                δ_e ← max{s − w(u, G'[H]), s − w(v, G'[H])};
                FR(e) ← |F(e, δ_test)|/δ_e;
                if FR(e) > max_FR then
                    e* ← e, δ* ← δ_e;
                    max_FR ← FR(e);
    if e* = none then
        break;
    w_G'(e*) ← w_G'(e*) + δ*;
    sum ← sum + δ*;
    A ← A ∪ {⟨e*, δ*⟩};
    C_s ← s-core(G');
return A
```

C Pseudo code of GPA

Algorithm description. Algorithm 3 presents the pseudocode of GPA, an accelerated variant of GIA that reduces the candidate space while preserving the same greedy selection criterion. GPA exploits the locality of follower propagation by organising nodes outside the s-core into connected components and maintaining, for each component, the best candidate edge–weight increment pair identified so far. At each iteration, GPA selects the current best candidate across all components. It then explores candidate edges whose endpoints belong to different components, guided by node-level and edge-level upper bounds that allow early termination and safe pruning. Only candidate pairs that cannot be ruled out by these bounds are evaluated using the same follower computation and MEI calculation as in GIA. After applying the selected edge–increment action, GPA updates the affected components and their best candidates while reusing results from unchanged components when feasible. If a cached candidate exceeds the remaining budget, its component is re-evaluated. The s-core is updated locally within the affected regions. This process repeats until the budget is exhausted or no feasible positive-gain candidate remains, yielding the same greedy outcome as GIA with fewer candidate evaluations.

Algorithm 3: GPA(G, b, s)

$G' \leftarrow G, A \leftarrow \emptyset, sum \leftarrow 0$;
$C_s \leftarrow s\text{-core}(G')$;
$M \leftarrow$ `buildComponents`(G', b, s);
while $sum < b$ **do**
  $L \leftarrow$ `list()`;
  $e^* \leftarrow none, \delta^* \leftarrow 0, max_FR \leftarrow 0$;
  if $M \neq \emptyset$ **then**
    $(e^*, \delta^*, max_FR) \leftarrow \arg\max_{(e,\delta_e,FR)\in M} FR$;
  for $u \in V \setminus C_s$ *and* $sum + (s - c(u)) \leq b$ **do**
    $U(u) \leftarrow$ `UpperBound`(G', u, s);
    `L.append(u)`;
  `sort(L)` ; // by upper bound decreasing order
  for $u \in L$ **do**
    if $max_FR > 2 \cdot U(u)$ **then**
      break;
    for $v \in L$ *s.t. v appears after u* **do**
      if *u.component* = *v.component* **then**
        continue;
      if $max_FR > U(u) + U(v)$ **then**
        break;
      if $max_FR > U(u, v)$ **then**
        continue;
      $e \leftarrow (u, v)$;
      $\tilde{\delta}_e \leftarrow s - \min\{c(u), c(v)\}$;
      if $sum + \tilde{\delta}_e \leq b$ **then**
        $\overline{\delta}_e \leftarrow s - w_{G'}(e)$;
        $\delta_{\text{test}} \leftarrow \min\{\overline{\delta}_e, b - sum\}$;
        $\mathcal{F}(e, \delta_{\text{test}}) \leftarrow$ `Followers`$(G', e, \delta_{\text{test}}, s, C_s)$;
        if $\mathcal{F}(e, \delta_{\text{test}}) \neq \emptyset$ **then**
          $H \leftarrow C_s \cup \mathcal{F}(e, \delta_{\text{test}})$;
          $\delta_e \leftarrow \max\{s - w(u, G'[H]), s - w(v, G'[H])\}$;
          $FR(e) \leftarrow |\mathcal{F}(e, \delta_{\text{test}})|/\delta_e$;
          if $max_FR < FR(e)$ **then**
            $e^* \leftarrow e, \delta^* \leftarrow \delta_e$;
            $max_FR \leftarrow FR(e)$;
  if $e^* = none$ **then**
    break;
  $w_{G'}(e^*) \leftarrow w_{G'}(e^*) + \delta^*$;
  $sum \leftarrow sum + \delta^*$;
  $A \leftarrow A \cup \{\langle e^*, \delta^* \rangle\}$;
  `updateComponents`(G', b, s, sum, M);
return A

D Proof of Theorem 1

Proof. We reduce the Maximum Coverage (MC) problem, which is known to be NP-hard unless P = NP [1], to the WASCO problem. The MC problem is defined as follows: given a collection of sets $\{T_1, T_2, \ldots, T_t\}$ containing elements from a universe $\{e_1, e_2, \ldots, e_d\}$ and a budget b', the objective is to select b' sets such that the number of unique elements covered is maximised.

To construct an instance of the WASCO problem, consider an arbitrary instance I_{MC} of the MC problem with sets $\{T_1, \ldots, T_t\}$ and elements $\{e_1, \ldots, e_d\} = \bigcup_{i=1}^{t} T_i$, as illustrated in the top of Figure 1. We define an instance $I_{\text{WASCO}} = (G, s, b)$ as follows:

- The graph G consists of three components: a part M representing the sets T_i, a part N representing the elements e_j, and a set of $(d+2)$-cliques.
- For each set T_i, create a corresponding set of nodes M_i in M, where $M_i = \{u_{i,1}, u_{i,2}, \ldots, u_{i,d+2}\}$. The nodes $\{u_{i,1}, \ldots, u_{i,d+1}\}$ form a $(d+1)$-clique, and $u_{i,d+2}$ is connected to $\{u_{i,1}, \ldots, u_{i,d}\}$.

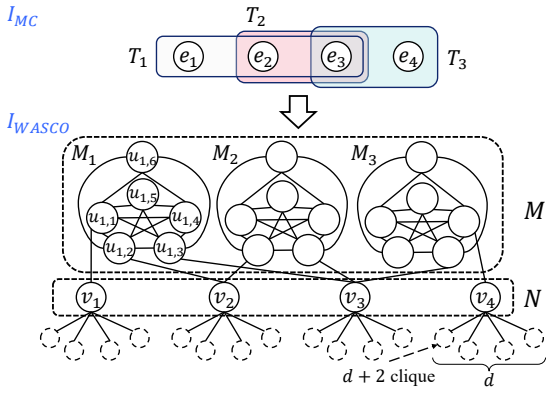


Figure 1: Proof of NP-hardness

- For each element e_j, create a corresponding node v_j in N. Add an edge between v_j and $u_{i,j}$ for every set T_i such that $e_j \in T_i$.
- For each v_j, add d disjoint $(d+2)$-cliques, and connect v_j to one node in each clique.
- Set $s = d + 1$ and $b = b'$.
- Every edge in G has a weight of 1.

This construction can clearly be completed in polynomial time. Initially, during the s-core decomposition, all nodes in M and N are removed, leaving only the $(d+2)$-cliques. This happens because:

- Nodes $u_{i,d+1}$ and $u_{i,d+2}$ in M_i have degree d, which is less than $s = d + 1$, and are therefore removed.
- Removing $u_{i,d+1}$ and $u_{i,d+2}$ causes the remaining nodes $u_{i,j}$ in M_i to fail the s-core condition, as their degree drops below s.
- All v_j nodes in N are similarly removed because their connections to M_i are lost.

As a result, only the $(d+2)$-cliques remain in the s-core after this initial decomposition. To maximise the s-core size, an optimal solution is to add an edge between $u_{i,d+1}$ and $u_{i,d+2}$ in M_i. Adding this edge corresponds to selecting the set T_i in the MC problem. Note that another optimal solution adds edges between distinct sets in M (e.g., $(u_{i,d+2}, u_{j,d+1})$ and $(u_{j,d+2}, u_{i,d+1})$), which can always be replaced by the intra-set connection since both strategies yield the exact same number of followers. When the edge $(u_{i,d+1}, u_{i,d+2})$ is added, this single edge brings the entire M_i together with all of its adjacent v_j nodes in N into the s-core:

- All nodes in M_i are included, since the degrees of $u_{i,d+1}$ and $u_{i,d+2}$ become $d + 1$.
- All v_j connected to M_i are included as well, as their degree conditions are now satisfied.

We now argue that this connection is optimal. Our strategy gains an entire M_i and its adjacent v_j nodes at the cost of a single action, so to outperform it, an alternative must achieve a larger gain at the same cost. We therefore compare every candidate from the viewpoint of the gain a single weight increment produces. Recall that, without the edge $(u_{i,d+1}, u_{i,d+2})$, a single unit can lift at most one of the two deficient nodes $u_{i,d+1}$ and $u_{i,d+2}$ to weighted degree $d + 1$, so the other is removed. After this removal, each $u_{i,j}$ $(j \leq d)$ that is not an endpoint of the reinforced edge has weighted degree d within M_i, and thus survives only if it is supported by v_j, i.e., only if $e_j \in T_i$.

Hence M_i can survive only partially, and only under this condition. We thus examine every way of spending one unit of weight:

- **Raising the weight of an existing edge between $u_{i,d+1}$ (or $u_{i,d+2}$) and some $u_{i,j}$ with $j \le d$ by 1.** If $\{e_k : k \le d,\ k \ne j\} \subseteq T_i$, the followers are M_i minus the unlifted deficient node together with $\{v_k : e_k \in T_i\}$, i.e., $d + 1 + |T_i|$ nodes; otherwise, M_i collapses and the gain is 0. In both cases, the follower set is a proper subset of that of $(u_{i,d+1}, u_{i,d+2})$, which yields $d + 2 + |T_i|$ followers.
- **Raising the weight of an edge $(u_{i,j}, u_{i,k})$ with $j, k \le d$.** Both deficient nodes are removed. For $d \ge 3$, the collapse propagates and the gain is 0; for $d = 2$, $u_{i,1}$ and $u_{i,2}$ survive only if $T_i = \{e_1, e_2\}$, giving $\{u_{i,1}, u_{i,2}, v_1, v_2\}$, again a proper subset of the followers of $(u_{i,d+1}, u_{i,d+2})$.
- **Cross connections between distinct M_i and M_j** (e.g., adding weight on $(u_{i,d+2}, u_{j,d+1})$ and $(u_{j,d+2}, u_{i,d+1})$). This can revive both M_i and M_j, but it spends two units—exactly the same budget as the two intra connections $(u_{i,d+1}, u_{i,d+2})$ and $(u_{j,d+1}, u_{j,d+2})$—and revives the same M_i, hence the same v_j nodes. It can therefore always be replaced by intra connections with no loss. However, cross connections to a node in $\{u_{\cdot,1}, \dots, u_{\cdot,d}\}$ are inferior for the same reason as the first case.
- **Connecting a node in N or in a $(d+2)$-clique to $u_{i,d+1}$ or $u_{i,d+2}$.** Again only one deficient node is lifted. The remaining nodes of M_i survive only if $T_i = \{e_1, \dots, e_d\}$, in which case the followers are M_i minus the other deficient node together with $\{v_1, \dots, v_d\}$, a proper subset of the followers of $(u_{i,d+1}, u_{i,d+2})$; otherwise, the gain is 0.
- **Adding weight within N (i.e., on (v_j, v_k)).** Since each v_j is already connected to d of the $(d+2)$-cliques, it only needs one more unit of weighted degree to survive; thus a single unit on (v_j, v_k) can bring two nodes v_j and v_k into the s-core. However, this gain of 2 is far smaller than the gain of our strategy, so it is not optimal.
- **Adding weight to an edge between v_j and a clique node.** v_j reaches weighted degree $d + 1$ and joins the s-core, but no M_i is revived since its deficient nodes remain below s; the gain is exactly 1. Any other edge involving a clique node yields gain 0.

Therefore, every single-unit action other than $(u_{i,d+1}, u_{i,d+2})$ either yields at most two followers or yields a follower set strictly contained in that of $(u_{i,d+1}, u_{i,d+2})$ for the corresponding i, and can thus be replaced by the latter without loss. Hence, an optimal solution spends each of its b units to complete a distinct M_i. This corresponds exactly to selecting the set T_i in the MC problem. The resulting s-core size gain equals

$$\underbrace{(d+2)\cdot b}_{\text{nodes of the selected } M_i} + \left|\{\, v_j : e_j \in \textstyle\bigcup_{i \in S} T_i \,\}\right|,$$

where S is the set of selected M_i with $|S| = b$. The first term is constant for any solution using the full budget, while the second counts the distinct elements covered. Maximising the s-core size thus reduces exactly to maximising the number of distinct covered elements—the objective of the MC problem.

Since the optimal way to maximise the s-core is to add edges corresponding to specific T_i sets so as to include as many e_i elements as possible in the s-core, this process directly mirrors the objective of the MC problem to select sets that maximise the coverage of elements. Thus, selecting M_i that are more densely connected to uncovered v_j aligns with the optimisation goals of the MC problem.

Therefore, the MC problem is reducible to the WASCO problem, implying that $I_{MC} \le_P I_{\mathsf{WASCO}}$. As the MC problem is NP-hard, this proves the NP-hardness of the WASCO problem. □

E Proof of Theorem 2

Proof. Suppose that f is submodular. Then, for any two sets A and B, it must hold that $f(A)+f(B) \ge f(A\cup B)+f(A\cap B)$. Consider a graph consisting of four nodes $\{u, v, x, y\}$ and two edges (u, x) and (v, y), each with a weight of 1. Let $s = 2$, If $A = \{\langle (u, v), 1\rangle\}$ and $B = \{\langle (x, y), 1\rangle\}$, $f(A) + f(B) = 0 < f(A \cup B) + f(A \cap B) = 4$. □

F Proof of Theorem 3

Proof. Let $G_{e,\delta}$ denote the graph obtained by anchoring e with δ. Fix the peeling order used to compute $C_s(G)$, with ties inside the same coreness–layer broken arbitrarily but deterministically. Suppose, for contradiction, that there exists a follower $x \in C_s(G_{e,\delta}) \setminus C_s(G)$, $x \notin \{u, v\}$, that is not reachable from either endpoint by an upstair path, and choose such an x with the smallest peeling order. Since $x \notin C_s(G)$, when x is removed in the peeling process of G, its weighted support from nodes not removed before x is less than s. Since $x \in C_s(G_{e,\delta})$, however, x has support at least s in $C_s(G_{e,\delta})$. As x is not an endpoint of e, the anchoring operation does not directly change any edge incident to x; hence the additional support must come from some neighbour y that was removed before x in G but is retained in $G_{e,\delta}$, i.e., $y \prec x$ and $y \in C_s(G_{e,\delta}) \setminus C_s(G)$. Thus y is also a follower. By the minimal choice of x, y is reachable from u or v by an upstair path. Since $(y, x) \in E$ and $y \prec x$, appending x to this path gives an upstair path to x, a contradiction. Therefore, every follower is reachable from an endpoint of e by an upstair path. □

G Proof of Theorem 4

fix되면 그대로 들어올 예정

H Proof of Theorem 5

fix되면 그대로 들어올 예정

I Proof of Theorem 6

Proof. Let $e = (u, v)$ and $e' = (u, x)$, where $u \notin C_s(G)$ and $v, x \in C_s(G)$. Consider any positive increment δ and let $K = C_s(G_{e,\delta})$. Since anchoring cannot shrink the s-core, $C_s(G) \subseteq K$, so both v and x belong to K.

We show that K also satisfies the s-core constraint in $G_{e',\delta}$. Every node in $C_s(G)$ already satisfies the constraint within $G[C_s(G)]$ and therefore remains valid. For nodes in $K \setminus C_s(G)$ other than u, the weighted degrees remain unchanged regardless of which edge is anchored. Moreover, since $v, x \in K$, anchoring either edge by δ provides the same additional support to u, i.e.,

$$w(u, G_{e,\delta}[K]) = w(u, G[K]) + \delta = w(u, G_{e',\delta}[K])$$

Thus, every node in K satisfies the s-core constraint in $G_{e',\delta}$, implying

$$K \subseteq C_s(G_{e',\delta})$$

By exchanging e and e', the reverse inclusion also holds. Hence, $C_s(G_{e,\delta}) = C_s(G_{e',\delta})$ for every positive increment δ.

Consequently, both edges induce identical follower sets for every positive increment δ. Therefore, the minimum increment that induces followers is also identical, giving the same MEI value $\delta_e = \delta_{e'}$. □

J Proof of Property 1

Ties in follower ratio are broken by a fixed total order $\prec_{\text{tb}}$, the lexicographic order of the canonical pairs $(\min\{u,v\}, \max\{u,v\})$. Both GIA and GPA rank candidates by the *greedy order* $\triangleright$: $e \triangleright e'$ if $FR(e) > FR(e')$, or $FR(e) = FR(e')$ and $e \prec_{\text{tb}} e'$. As follower ratios are compared exactly, $\triangleright$ is a strict total order: every finite candidate set has a unique $\triangleright$-maximum, regardless of scan order. For the current graph G_t and remaining budget r_t, the *feasible* candidates are

$$\mathcal{E}_t = \{e = (u,v) : u \neq v,\ \{u,v\} \nsubseteq C_s(G_t),\ \delta_e \leq r_t\},$$

where δ_e is the MEI of e in G_t.

Candidate evaluation. Both algorithms evaluate candidates by the procedure of Section V-B. By Theorems 4 and 5, it discards e if and only if $\delta_e > r_t$, and otherwise returns exactly δ_e and $\mathcal{F}(e, \delta_e)$. Although $\delta_{\text{test}}(e)$ depends on r_t, the returned values do not: the budget matters only through $\delta_e \leq r_t$, and δ_e, $\mathcal{F}(e, \delta_e)$, and $FR(e)$ depend on G_t alone. GIA selects the $\triangleright$-maximum of $\mathcal{E}_t$ and terminates when $\mathcal{E}_t = \emptyset$, possibly with $r_t > 0$.

PROOF. We show by induction on t that GPA and GIA enter iteration t in the same state (G_t, r_t) and then either both terminate or both select the same edge–increment pair.

Base case. Both start with $(G_1, r_1) = (G, b)$, hence the same $C_s(G)$, coreness–layer pairs, $\tilde{\delta}_u$, $P(u)$, and $U(u)$.

Inductive step. Assume both algorithms selected the same edge–increment pairs in the first $t-1$ iterations; then they share G_t, r_t, $C_s(G_t)$, the coreness–layer pairs, and $\mathcal{E}_t$. Let e^* be the $\triangleright$-maximum of $\mathcal{E}_t$, which GIA selects. To show that GPA also selects e^*, we establish two properties: (i) every candidate retained by GPA for selection is feasible and has the exact MEI and follower ratio; and (ii) GPA never discards e^*.

We prove these properties through candidate reduction, upper-bound pruning, and intermediate-result reuse (Items 1–3 below show). Let ρ be the follower ratio of the best candidate held by GPA during iteration t ($\rho = 0$ initially). Since every retained candidate is feasible, $\rho \leq FR(e^*)$ whenever $\mathcal{E}_t \neq \emptyset$.

(1) **Candidate reduction.** GPA considers only nodes $u \notin C_s(G_t)$ with $\tilde{\delta}_u \leq r_t$; otherwise every candidate e at u has $\delta_e \geq \tilde{\delta}_e \geq \tilde{\delta}_u > r_t$ by Theorem 4, so $e \notin \mathcal{E}_t$. If both endpoints of e^* lie outside $C_s(G_t)$, it is unaffected; otherwise let $e^* = (u,v)$ with $v \in C_s(G_t)$. By Theorem 6, all candidates in $\mathcal{T}_u = \{(u,x) : x \in C_s(G_t)\}$ share the MEI and follower set of e^*, hence its follower ratio and feasibility; as the $\triangleright$-maximum of $\mathcal{E}_t \supseteq \mathcal{T}_u$, e^* is the $\prec_{\text{tb}}$-minimal member of $\mathcal{T}_u$. This is the representative (u, z_t) kept by GPA, with z_t the smallest identifier in $C_s(G_t)$, since $(u,x) \prec_{\text{tb}} (u,x')$ whenever $x < x'$. At each iteration, GPA sets z_t to the smallest identifier in $C_s(G_t)$ to preserve tie-breaking.

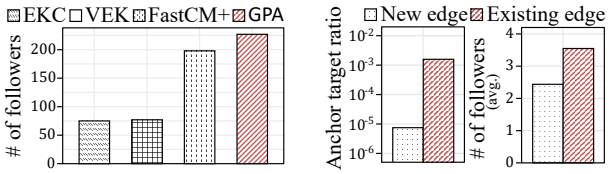
(2) **Upper-bound pruning.** By Theorems 3 and 4 and Definition 13, every feasible candidate $e = (u,v)$ with both endpoints outside $C_s(G_t)$ satisfies $FR(e) \leq U(u,v) \leq U(u) + U(v)$. For a representative $e = (u, z_t)$, $FR(e) \leq U(u)$.
GPA discards a candidate only if ρ *strictly* exceeds a valid bound on its follower ratio: $U(u,v)$ for candidate skipping; $U(u)+U(v)$ for partner-level termination, valid for every later partner v' since partners are scanned in descending $U(\cdot)$ ($U(u,v') \leq U(u) + U(v') \leq U(u) + U(v)$); and $2 \cdot U(u)$ for global termination, since later nodes have bounds at most $U(u)$. If e^* were discarded with bound B, then $FR(e^*) \leq B < \rho \leq FR(e^*)$, a contradiction. Strictness keeps candidates that may tie ρ and win under $\prec_{\text{tb}}$; the scan order only affects how fast ρ grows. The same holds for the pruned search within a region (item 3).

(3) **Intermediate-result reuse.** GPA maintains regions of non-s-core nodes, initially the connected components of $G[V \setminus C_s(G)]$. After each selection, the regions containing an endpoint of the selected edge are merged and re-evaluated. Thus, every connected component of $G_t[V \setminus C_s(G_t)]$ remains within a region. For each region R, let Ω_R contain its internal candidates and representatives (u, z_t), and let c_R be the $\triangleright$-maximum cached edge when R was last searched at iteration $t' \leq t$. All other candidates are inter-region.
Locality. By Theorem 3, followers of a selected edge remain within the regions containing its non-s-core endpoints. Thus, an unaffected region R retains its internal structure and connections to the s-core; otherwise, a newly engaged neighbour would have belonged to the same non-s-core component. By Theorem 5, the MEI, follower set, and follower ratio of every candidate in Ω_R remain unchanged. If z_t changes, GPA re-evaluates all regions to preserve tie-breaking.
Shrinking budget. Since $r_t \leq r_{t'}$, some cached candidates may become infeasible. If $\delta_{c_R} \leq r_t$, the cached c_R remains the best feasible candidate in R and can be reused. Otherwise, R must be re-searched to find its current best candidate. However, this search can be deferred if another feasible cached candidate $c \triangleright c_R$ exists, since c_R still outranks every candidate in R. Thus, deferring the search cannot exclude the globally best feasible candidate from consideration.
Preservation of e^*. If $e^* \in \Omega_R$, it is either the valid cached maximum or is found by re-searching R. In the latter case, re-evaluation cannot be deferred, since no feasible candidate outranks e^*. Otherwise, e^* is inter-region and is evaluated afresh; by Item 2, it cannot be pruned. Thus, e^* reaches the final comparison under $\triangleright$, establishing completeness.

If $\mathcal{E}_t = \emptyset$, both algorithms terminate. Otherwise, Items 1–3 establish that GPA retains e^* with its exact MEI and follower ratio, even when retrieved from the cache. Thus, both algorithms select the same edge–increment pair, yielding identical G_{t+1} and $r_{t+1} = r_t - \delta_{e^*}$. By Definition 5 and the locality established above, GPA's local s-core update agrees with GIA's global recomputation. Hence, by induction, both algorithms select identical edge–increment pairs, terminate together, and return the same final s-core. □

K Additional Experiment

EQ1-4. Effectiveness (unweighted graph). We further evaluate GPA on the unweighted Facebook dataset [2] ($|V| = 4,039$, $|E| = 88,234$, $d_{\text{avg}} = 43.7$). All compared methods, including EKC [4], VEK [5], and FastCM+ [3], are originally designed for unweighted graphs. To enable a fair comparison, we treat the unweighted graph as a special case of our weighted formulation by assigning unit weight to every edge, and follow the same parameter settings ($k = 20$,



(a) $k = 20$, $b = 200$ **(b) Anchored edge analysis**

Figure 2: EQ1-4. Effectiveness on the Facebook

$b = 200$) as in FastCM+. As shown in Figure 2(a), GPA can achieve the largest number of followers among all compared methods under the same budget. This result indicates that, even when restricted to the unweighted setting, GPA is capable of operating effectively in practice. Figure 2(b) further analyses the anchoring behaviour of GPA. After normalising by the size of the corresponding feasible action spaces, edge reinforcement is selected more frequently than new edge insertion. Moreover, reinforcement actions tend to yield a higher average number of followers per anchored edge. These observations suggest that strengthening existing connections can be a more effective anchoring strategy than edge insertion, even when all edges have uniform weight.

References

[1] Samir Khuller, Anna Moss, and Joseph Seffi Naor. 1999. The budgeted maximum coverage problem. *Information processing letters* 70, 1 (1999), 39–45.

[2] Jure Leskovec and Julian Mcauley. 2012. Learning to discover social circles in ego networks. *Advances in neural information processing systems* 25 (2012).

[3] Xin Sun, Xin Huang, and Di Jin. 2022. Fast algorithms for core maximization on large graphs. *Proceedings of the Very Large Data Bases (VLDB) Endowment* 15, 7 (2022), 1350–1362.

[4] Zhongxin Zhou, Fan Zhang, Xuemin Lin, Wenjie Zhang, and Chen Chen. 2019. k-Core Maximization: An Edge Addition Approach.. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 4867–4873.

[5] Zhongxin Zhou, Wenchao Zhang, Fan Zhang, Deming Chu, and Binghao Li. 2022. VEK: a vertex-oriented approach for edge k-core problem. *World Wide Web* 25, 2 (2022), 723–740.